AMATH 582: HOMEWORK 5

CALEB CRAMER

Abstract. This report investigates the efficacy of spectral clustering and semi-supervised regression algorithms applied to the 1984 United States Congressional Voting Records dataset. The dataset comprises the voting records of 435 House members across 16 distinct bills. Traditional unsupervised methods often fail to capture the complex, non-linear partisan alignments present in such categorical data. By constructing a Graph Laplacian and projecting the data into a lower-dimensional spectral space, we extract structural connectivity features. These features are then utilized in a semi-supervised regression framework to predict party affiliation. Our computational results demonstrate the algorithms' ability to accurately identify partisan divides using minimal labeled data.

1. Introduction and Overview

The fundamental problem addressed in this report is the classification of 435 United States House members (267 Democrats and 168 Republicans) into their respective political parties based solely on their voting behavior across 16 key bills in 1984.

Traditional clustering algorithms often struggle with high-dimensional, categorical datasets where the underlying structures are non-convex and relationships are non-linear. To address this, relevant machine learning literature heavily favors spectral clustering methods. Rather than relying on rigid geometric distances, spectral methods treat the dataset as an interconnected network. By utilizing the eigenvalues and eigenvectors of a Graph Laplacian, spectral clustering performs dimensionality reduction, identifying topological bottlenecks in the graph that represent deep, structural partisan divides. These 'divides' are utilized to identify a Congressperson's official affiliation.

Furthermore, in many real-world classification scenarios, true labels are expensive or difficult to obtain. This report extends the unsupervised spectral approach by implementing a semi-supervised learning (SSL) regression algorithm. By leveraging the continuous spectral features derived from the network's structure alongside a small fraction of known party affiliations, we can propagate labels through the graph. This report details the theoretical foundation, algorithmic implementation, and computational results of applying these spectral and semi-supervised techniques to accurately reconstruct the political landscape of the 1984 House of Representatives.

2. Theoretical Background

2.1. **K-Means Clustering.** Traditional unsupervised learning methods, such as K-Means clustering, attempt to partition data by minimizing the variance within distinct groups. Given our dataset of voting records $x_1, \ldots, x_N \in \mathbb{R}^{16}$, the objective is to find a set of K clusters $S = \{S_1, \ldots, S_K\}$ and corresponding centroids $\mu_1, \ldots, \mu_K$ that minimize the within-cluster sum of squares:

$$\arg\min_{S} \sum_{i=1}^{K} \sum_{x_j \in S_i} \|x_j - \mu_i\|^2$$

Applied to the 1984 United States congressional voting dataset, this approach treats the 16 bills as a 16-dimensional coordinate space. However, K-Means assumes that clusters are convex and cleanly separable. It often struggles with high-dimensional, categorical data where relationships

Date: March 16, 2026.

between nodes are non-linear, making it difficult to capture complex political alliances that do not fall strictly along a straight geometric divide.

2.2. **Spectral Clustering.** To overcome the geometric limitations of algorithms like K-Means, spectral clustering relies on connectivity rather than absolute spatial coordinates. It treats the dataset as a graph $G = (V, E)$, where each House member is a node in V, and the edges E represent the similarity or alignment of their voting records. By analyzing the spectrum of this network, spectral clustering groups members based on their community connectedness [1].

2.2.1. *Graph Laplacian and Heat Diffusion.* The core of this method relies on the Graph Laplacian, a matrix that encodes the structure of the voting network. First, we construct a similarity matrix W, where each entry quantifies the voting alignment between two politicians. This is typically calculated using a Gaussian distance metric scaled by a tuning parameter η:

$$W_{ij} = \exp\left(-\eta \|x_i - x_j\|^2\right)$$

Two pedagogically useful examples of η are:

$$\eta(r) = \begin{cases} 1 & r \leq 1 \\ 0 & r > 1 \end{cases} \qquad \text{or} \qquad \eta(r) = e^{-\frac{1}{2}r^2} \tag{1}$$

where the first can be thought of an in/out requirement, drawing an edge if another point is within the radius. The second draws nearly all the edges but adjusts the weights according to the Gaussian evaluated at the radius, weaker edges for further points and vice versa.

Intuitively, this weight matrix is fundamentally linked to the physics of heat diffusion. Imagine the graph as a network of physical pipes; the weight W_{ij} represents the rate at which heat transfers between node i and node j. The parameter η acts as an inverse time or temperature decay constant, determining how quickly this heat dissipates over distance. If two voting records are very similar, the distance is small, heat flows freely, and the edge is mathematically strong. If they are dissimilar, the heat decays exponentially, resulting in a weak, disconnected/slow edge.

Next, we define the degree matrix D, which is a diagonal matrix where each entry represents the sum of the similarity weights for a given node:

$$D_{ii} = \sum_{j=1}^{N} W_{ij}$$

The unnormalized Graph Laplacian is then defined as:

$$L = D - W$$

A fundamental property of the Graph Laplacian is that it satisfies the following relation for any vector $f \in \mathbb{R}^N$:

$$f^T L f = \frac{1}{2} \sum_{i,j=1}^{N} W_{ij}(f_i - f_j)^2$$

This formulation highlights how L measures the smoothness of f across the graph, making it mathematically straightforward to identify the structural partisan divide between the Democrats and Republicans.

2.3. **Semi-Supervised Learning via Laplacian Embedding.** While spectral clustering provides a powerful unsupervised method for discovering community structure, semi-supervised learning (SSL) allows us to anchor these structural insights using a small subset of known labels. By treating the eigenvectors of the unnormalized Graph Laplacian as geometric features, we can construct a lower-dimensional embedding suitable for linear regression.

Given an integer $M \geq 1$, we define the Laplacian embedding for a specific data point $\mathbf{x}_j$ using the first M eigenvectors of the Laplacian matrix, denoted as $\mathbf{q}$. The new feature vector is given by:

$$F(\mathbf{x}_j) = ((\mathbf{q}_0)_j, (\mathbf{q}_1)_j, \ldots, (\mathbf{q}_{M-1})_j) \in \mathbb{R}^M$$

Compiling these vectors for all 435 House members yields the full Laplacian embedding matrix $F(X) \in \mathbb{R}^{435\times M}$, where the j-th row corresponds to the features of $F(\mathbf{x}_j)$. This matrix serves as our new, structurally-informed feature space.

To incorporate known affiliations, we select an integer parameter $J \geq 1$ representing the number of labeled politicians available to the algorithm. Preserving the original ordering of the dataset, we extract the first J rows of the embedding to construct a submatrix $A \in \mathbb{R}^{J\times M}$, alongside a target vector $\mathbf{b} \in \mathbb{R}^J$ containing the corresponding true labels $\mathbf{y}$. The elements are defined as:

$$A_{ij} = F(X)_{ij}, \quad i = 0, \ldots, J-1, \quad j = 0, \ldots, M-1$$
$$\mathbf{b}_i = \mathbf{y}_i, \quad i = 0, \ldots, J-1$$

With the features and targets established, we apply a linear regression (least squares) framework to learn the relationship between the graph's topology and the political parties. We seek the optimal coefficient vector $\hat{\boldsymbol{\beta}}$ that minimizes the residual sum of squares on the labeled subset:

$$\hat{\boldsymbol{\beta}} = \operatorname{argmin}_{\boldsymbol{\beta}\in\mathbb{R}^M} \|A\boldsymbol{\beta} - \mathbf{b}\|_2^2$$

Notably, because the first eigenvector $\mathbf{q}_0$ of an unnormalized Laplacian is a constant vector, it naturally functions as the intercept term (β_0) in the regression model.

Once the optimal weights $\hat{\boldsymbol{\beta}}$ are computed, the model can predict the classes of all points in the dataset X. By projecting the full embedding matrix onto the learned coefficients and applying a sign function to handle the binary classification boundary, we obtain our predicted labels:

$$\hat{\mathbf{y}} = \operatorname{sign}(F(X)\hat{\boldsymbol{\beta}})$$

The efficacy of this semi-supervised classifier is subsequently evaluated by computing the overall accuracy across the entire network—specifically, $1-\frac{1}{435}\times$(number of misclassified members)—allowing us to observe how variations in the embedding dimension M and the training size J impact predictive performance.

3. Algorithm Implementation and Development

The implementation of Spectral Clustering and SSL are far more simple than the sections in 2 would lead us to believe. The overall computational procedure can be broken down into a streamlined, seven-step process, which we detail below.

First, we preprocess the data. The raw 1984 congressional voting dataset contains categorical responses for the 16 bills (typically 'yea', 'nay', or 'unknown') which must be numerically encoded (e.g., 1, -1, and 0) to allow for mathematical operations. Once the 435 voting records are numerically represented in a matrix, we compute the distance matrix to establish the pairwise Euclidean distances between all members of the House.

With the distances established, the next step is to compute the weight matrix with η and the Graph Laplacian. We apply a kernel function, utilizing a tuning parameter η to control the decay of the weights, transforming the raw distances into a similarity weight matrix W. From

this, we construct the diagonal degree matrix D and compute the unnormalized Graph Laplacian, $L = D - W$.

We then find the eigenvalues and sort them in ascending order alongside their corresponding eigenvectors. The spectrum of the Laplacian reveals the fundamental connectivity of the graph. Specifically, using the 'Fiedler' vector, we sort the results. The Fiedler vector (the eigenvector corresponding to the second smallest eigenvalue) provides a one-dimensional embedding of the graph where the natural partisan divide between Democrats and Republicans becomes apparent. We have to be careful when evaluating the accuracy of our groupings because the clusters may correlate to a 1 or a -1 but they may be swapped from our original assignments. We check the accuracy of the as-is clusters and the flipped cluster to guard against this.

For the semi-supervised learning phase, we slice the eigenvalues with M and J. Here, M denotes the number of eigenvectors (the spectral features) we choose to retain to form a lower-dimensional representation of the voting data. Meanwhile, J represents the indices of the small subset of House members whose true party affiliations (the labels) are provided to the algorithm.

Finally, using regression, we fit and predict the results. By mapping the known party labels from the subset J to their corresponding continuous spectral features in our M-dimensional space, we train a simple regression model. This model is then used to extrapolate and predict the party affiliation of the remaining unknown House members, completing the semi-supervised classification task.

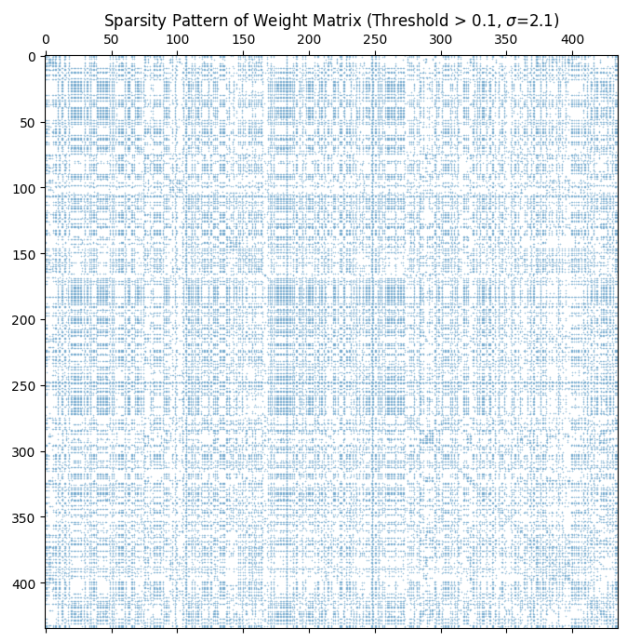

FIGURE 1. The Sparsity Matrix of the Weight Matrix

4. Computational Results

Figure 1 visualizes the structure of the unnormalized weight matrix, W, before any spectral sorting is applied. The 435×435 grid represents the pairwise similarities between all House members, where a plotted point indicates a voting affinity (edge weight) greater than the 0.1 threshold, calculated using a Gaussian kernel with $\sigma = 2.1$.

The distinct "plaid" or checkerboard pattern is highly revealing. The dense bands correspond to groups of politicians with highly correlated voting records across the 16 bills, representing strong

intra-party alignment. Conversely, the white, sparse regions represent pairs of politicians with fundamentally opposed voting records, resulting in weak or nonexistent edges in the network.

Notably, the fact that these dense blocks are scattered throughout the matrix rather than isolated along the main diagonal indicates that the raw dataset is not ordered by party affiliation. This visualization illustrates the core motivation for spectral clustering: the primary objective of computing the Graph Laplacian and extracting the Fiedler vector is to mathematically permute this exact matrix. By reordering the nodes based on the second eigenvector, spectral clustering untangles this scattered grid, shifting the dense affiliations into two distinct block-diagonal clusters that clearly expose the underlying partisan divide. This phenomenon can be seen in Figure 4!

Based on the eigenvalues of the Graph Laplacian (ignoring the 0 index), we can see this problem is ideal for splitting into two clusters as we have a massive jump between index 1 and index 2, as shown in Figure 2.

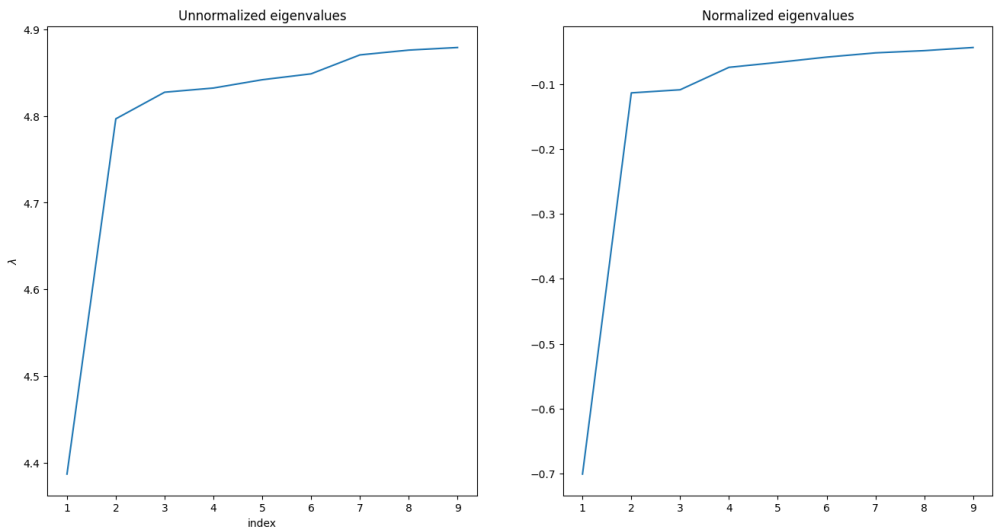

FIGURE 2. The sorted eigenvalues of the Graph Laplacian

4.1. **Spectral Clustering Results.** For the spectral clustering, we tuned the σ value with a loop through $(0, 4]$ and the η function described in Equation 1. We found the optimal sigma to be $\sigma = 1.2$ as shown in Figure 3.

4.2. **Semi-Supervised Clustering Results.** Then using the same σ from above, we attempt semi-supervised learning on the same dataset by labeling a few datapoints as an initialization seed for the algorithm. Some interesting results for both the unnormalized Laplacian and the Random Walk can be seen in Tables 1, 2, 3.

5. Summary and Conclusions

In conclusion, the 1984 voting dataset proved to be an excellent candidate for both spectral clustering and semi-supervised classification. Our results indicate that leveraging the network's underlying topology yields high predictive accuracy, which increases further as additional methods are introduced. Notably, while the unnormalized and Random Walk Graph Laplacians achieved highly similar results during the purely unsupervised spectral clustering phase, the Random Walk variant demonstrated superior performance when integrated into the semi-supervised learning framework. An interesting extension, in a similar vein to previous homework, would be using cross validation of the J selected Congresspeople during SSL. In our implementation, we only took the first J columns.

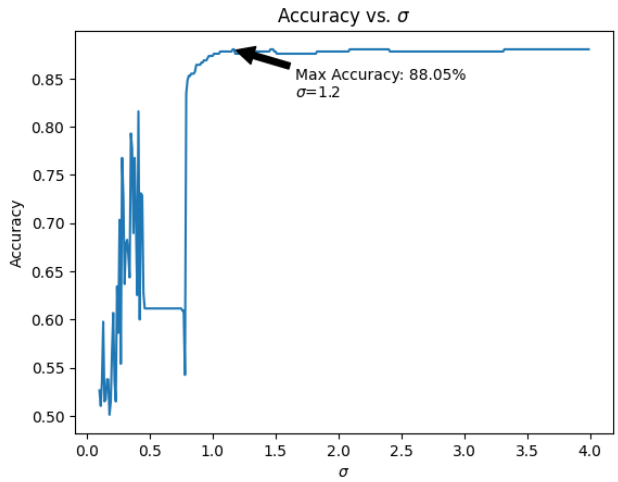

FIGURE 3. Searching for the highest accuracy among from $\sigma \in (0, 4]$

	Linear Regression $\sigma = 1.2$				
$J \setminus M$	2	3	4	5	6
5	0.6184	**0.8897**	0.8874	0.8805	0.8805
10	0.6345	0.8529	0.8506	0.8736	0.8736
20	0.8437	0.7655	0.8667	0.8736	0.8782
40	0.8713	0.6644	0.8368	0.8805	0.8805

	Ridge Regression $\sigma = 1.2$				
$J \setminus M$	2	3	4	5	6
5	0.8851	0.8851	0.8805	0.8828	0.8828
10	**0.8874**	0.8092	0.8276	0.8437	0.8460
20	0.8437	0.8552	0.8690	0.8782	0.8805
40	0.8690	0.7103	0.8368	0.8805	0.8828

TABLE 1. Accuracies for Semi-Supervised Learning using (Linear and Ridge Regression)

ACKNOWLEDGEMENTS

I am thankful to Artur, Ali, and Kyle for enlightening discussion, especially on the tuning of parameters. Assistance with LaTeX formatting was provided by Google and Gemini. I also heavily relied on Nathan Kutz's book, the lecture notes, and the `sklearn` documentation.

REFERENCES

[1] J. N. Kutz. Data-driven modeling and scientific computation methods for integrating dynamics of complex systems and big data.

Appendix A. Extra Tables and Figures

	Linear Regression $\sigma = 3.3$				
$J \setminus M$	2	3	4	5	6
5	0.8621	0.8920	0.5678	0.6230	0.8920
10	0.8575	0.8207	**0.9379**	0.6069	0.5126
20	0.8782	0.8989	0.9080	0.9057	0.9057
40	0.8782	0.8851	0.9149	0.9195	0.9011

	Ridge Regression $\sigma = 2.9$				
$J \setminus M$	2	3	4	5	6
5	0.8713	0.8989	0.5816	0.5839	0.5885
10	0.8713	0.8575	**0.9333**	0.7839	0.6161
20	0.8759	0.9011	0.9149	0.9057	0.8897
40	0.8805	0.8805	0.9034	0.9103	0.9057

Table 2. Comparison of SSL Accuracies for the best σ

	Linear Regression $\sigma = 1.33$				
$J \setminus M$	2	3	4	5	6
5	0.5402	0.6161	0.5195	0.5954	0.5931
10	0.8644	0.8851	0.8897	0.8552	0.7977
20	0.8805	0.8966	0.8989	0.8644	0.8690
40	0.8966	0.8943	0.8943	**0.9402**	0.9126

	Ridge Regression $\sigma = 1.33$				
$J \setminus M$	2	3	4	5	6
5	0.5402	0.6161	0.5264	0.5908	0.5931
10	0.8644	0.8851	0.8897	0.8552	0.7977
20	0.8805	0.8966	0.8989	0.8644	0.8713
40	0.8966	0.8943	0.8943	**0.9402**	0.9126

Table 3. Comparison of SSL Accuracy for the Random Walk Graph with best σ

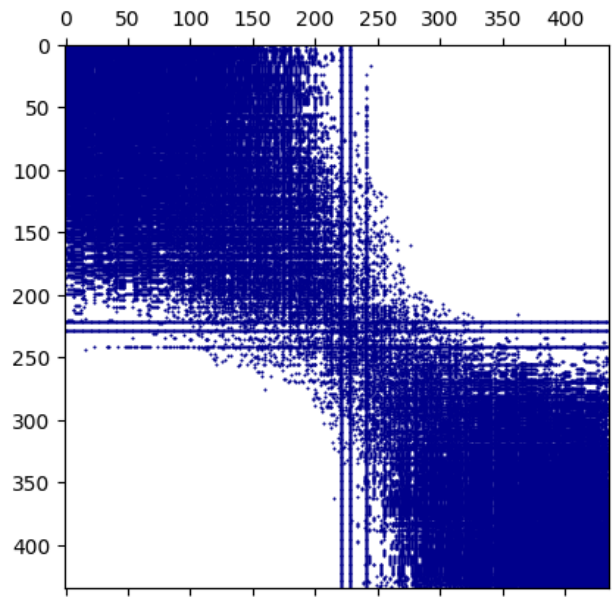

FIGURE 4. Using the Fiedler Vector, we are able to sort the weight matrix to see the natural split between the parties.

APPLIED MATHEMATICS, UNIVERSITY OF WASHINGTON, SEATTLE, WA
Email address: cbcramer@uw.edu